Compte rendu de projet

API Iris conteneurisée avec mémoire Redis

Étudiant : Aymane El Yamani
Professeur : Mohamed Ameksa

Année universitaire 2026

15 octobre 2026

Table des matières

1 Objectif du projet **2**

2 Architecture **2**

3 Implémentation de l'API **2**
3.1 Endpoints 3

4 Conteneurisation **3**

5 Orchestration avec Docker Compose **3**

6 Tests et résultats **4**
6.1 Effet de `docker compose down` 4
6.2 Effet de `docker compose down -v` 4

7 Bilan **5**

1 Objectif du projet

L'objectif du projet est de reprendre une API de prédiction basée sur le jeu de données Iris et de lui ajouter une mémoire. L'API effectue les prédictions, tandis que Redis compte le nombre total de prédictions réalisées.

Le projet met en œuvre :

— une API HTTP développée avec FastAPI ;

— un modèle de régression logistique entraîné sur Iris ;

— une communication entre conteneurs avec Docker Compose ;

— un compteur persistant dans Redis grâce à un volume nommé.

2 Architecture

L'application est composée de deux services :

`api` Conteneur Python qui expose l'API sur le port 8000. Le modèle Iris est chargé et entraîné au démarrage du processus.

`base`
Conteneur Redis basé sur l'image `redis:7-alpine`. Le mode AOF est activé afin de conserver les écritures sur le volume `redis-data`.

Le nom DNS `base` est utilisé par l'API pour joindre Redis sur le port 6379. Le service `api` dépend du healthcheck Redis et ne démarre qu'une fois que Redis répond correctement à la commande `PING`.

3 Implémentation de l'API

Le fichier principal est `src/app.py`. Le modèle est entraîné à partir de `load_iris` avec une régression logistique :

```
iris = load_iris()
model = LogisticRegression(max_iter=300, random_state=42)
model.fit(iris.data, iris.target)
```

La requête de prédiction doit contenir exactement quatre mesures :

```
class PredictionRequest(BaseModel):
    features: Annotated[
        list[float],
        Field(min_length=4, max_length=4),
    ]
```

Lorsqu'une prédiction réussit, l'API appelle `INCR` sur la clé Redis `predictions`. Cette opération est atomique et renvoie directement la nouvelle valeur du compteur.

3.1 Endpoints

Méthode	Route	Rôle
GET	/health	Vérifie que Redis est accessible
POST	/predict	Prédit une espèce et incrémente le compteur
GET	/stats	Retourne le nombre de prédictions

Exemple de requête :

```
curl -X POST http://localhost:8000/predict ^
  -H "Content-Type: application/json" ^
  -d "{\"features\":[5.1,3.5,1.4,0.2]}"
```

Réponse observée :

```
{"prediction":0,"species":"setosa","count":1}
```

La route /stats renvoie ensuite :

```
{"predictions":1}
```

Si Redis est indisponible, l'API retourne une erreur HTTP 503 au lieu de simuler un compteur ou de masquer l'erreur.

4 Conteneurisation

Le Dockerfile utilise python:3.12-slim. Les dépendances sont installées depuis requirements.txt, puis seul le fichier src/app.py est copié dans l'image. L'application s'exécute avec l'utilisateur non privilégié appuser.

La commande de démarrage est :

```
uvicorn app:app --host 0.0.0.0 --port 8000
```

Le nom app:app correspond au fichier app.py et à l'objet FastAPI nommé app.

5 Orchestration avec Docker Compose

Le fichier docker-compose.yml définit les deux services et le volume :

```
services:
  api:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "8000:8000"
    depends_on:
      base:
        condition: service_healthy

  base:
    image: redis:7-alpine
```

```
    command: redis-server --appendonly yes
    volumes:
      - redis-data:/data
```

Avant le démarrage, la configuration a été vérifiée avec :

```
docker compose config
```

Cette commande a confirmé que les services, le healthcheck, le réseau et le volume étaient correctement interprétés par Compose.

6 Tests et résultats

L'application a été construite et démarrée avec :

```
docker compose up --build
```

Le service Redis a répondu `PONG` à son healthcheck, puis Uvicorn a indiqué que l'API écoutait sur `0.0.0.0:8000`.

Trois requêtes identiques sur `/predict` ont renvoyé successivement les compteurs 1, 2 et 3. Le résultat de `/stats` était :

```
{"predictions":3}
```

6.1 Effet de `docker compose down`

La commande :

```
docker compose down
docker compose up -d
```

supprime les conteneurs et le réseau, mais conserve le volume nommé. Après le redémarrage, `/stats` a donc renvoyé :

```
{"predictions":3}
```

Le compteur a bien survécu à l'arrêt normal de l'application.

6.2 Effet de `docker compose down -v`

La commande :

```
docker compose down -v
docker compose up -d
```

supprime également le volume `redis-data`. Redis redémarre alors avec une base vide et `/stats` renvoie :

```
{"predictions":0}
```

7 Bilan

Le projet répond à l'objectif demandé : la prédiction reste assurée par l'API et Redis fournit une mémoire externe pour compter les appels. La persistance dépend directement de la durée de vie du volume Docker :

— `down` conserve les données ;

— `down -v` les supprime.

La séparation entre l'API et Redis permet également de redémarrer le service web sans perdre l'état du compteur, tout en conservant une configuration reproductible avec Docker Compose.